

DOKUMENTACJA TECHNICZNA

Platforma Komunikacyjna

Wersja 1.0 | Maj 2026

Dokument poufny — przeznaczony dla partnerów biznesowych

Wszelkie prawa zastrzeżone | 2026

SPIS TRESCI

1. Architektura systemu
2. Stack technologiczny
3. Schemat bazy danych
4. API REST — endpointy
5. Komunikacja real-time (Socket.IO)
6. Wymagania serwerowe
7. Provisioning — flow automatyczny
8. Bezpieczeństwo
9. Skalowanie
10. Monitoring i dostępność

1. ARCHITEKTURA SYSTEMU

1.1 Przegląd architektury

System składa się z dwóch warstw:

Warstwa zarządzania (operator platformy):

- Panel administracyjny do zarządzania klientami i subskrypcjami
- System billing i płatności
- API provisionowania (automatyczne tworzenie serwerów klientów)
- Frontend aplikacji (Vercel CDN — współdzielony)

Warstwa klienta (dedykowana per klient):

- Serwer VPS z pełnym stackiem aplikacyjnym
- Izolowana baza danych MySQL
- Serwer głosowy LiveKit SFU
- Serwer TURN (coturn) dla NAT traversal

1.2 Diagram architektury



2. STACK TECHNOLOGICZNY

2.1 Frontend

Technologia	Wersja	Zastosowanie
Next.js	14.2	Framework React, SSR/SSG
TypeScript	5.x	Typowanie statyczne
Tailwind CSS	3.x	Stylowanie komponentów
Zustand	4.5	Zarządzanie stanem aplikacji
Socket.IO Client	4.7	WebSocket komunikacja real-time
LiveKit Client SDK	2.5	Głos i wideo (WebRTC)

2.2 Backend

Technologia	Wersja	Zastosowanie
Node.js	20 LTS	Srodowisko uruchomieniowe
Express	4.x	HTTP API REST
TypeScript	5.x	Typowanie statyczne
Socket.IO	4.7	Real-time WebSocket
MySQL2	3.x	Sterownik bazy danych
LiveKit Server SDK	2.x	Zarzadzanie pokojami głosowymi
JWT	9.x	Autentykacja tokenowa
bcrypt	5.x	Hashowanie hasel
multer	1.x	Upload plikow

2.3 Infrastruktura

Technologia	Zastosowanie
MySQL 8.0	Relacyjna baza danych
LiveKit SFU	Selective Forwarding Unit (glos/wideo)
coturn	Serwer TURN/STUN dla NAT traversal
Docker + Compose	Konteneryzacja calego stacku klienta
nginx	Reverse proxy, SSL termination
Let's Encrypt	Automatyczne certyfikaty SSL

2.4 Aplikacja desktopowa

Technologia	Zastosowanie
Electron 31	Wrapper desktopowy (Windows)
electron-updater	Automatyczne aktualizacje aplikacji
uihook-napi	Globalne skroty klawiszowe (PTT)
electron-builder	Budowanie instalatora NSIS

3. SCHEMAT BAZY DANYCH

3.1 Glowne tabele

Tabela	Opis
--------	------

users	Konta uzytkownikow, autentykacja, profile, statusy
servers	Serwery komunikacyjne (plany: free/standard/pro)
server_members	Czlonkostwo uzytkownikow w serwerach
roles	Role z granularnym systemem uprawnień (JSON)
channels	Kanaly (text/voice/announcement/forum/stage)
messages	Wiadomosci z obsuga watkow i odpowiedzi
reactions	Reakcje emoji na wiadomosci
polls / poll_options / poll_votes	Kompletny system ankiet
message_attachments	Zalaczniki plikow i obrazow
notifications	Powiadomienia (wzmianki, odpowiedzi, reakcje)
voice_states	Aktualny stan uzytkownikow na kanalach glosowych

3.2 Indeksy wydajnosciowe

Indeks	Tabela / Kolumny	Cel
idx_channel_created	messages(channel_id, created_at DESC)	Stronicowanie wiadomosci
idx_reactions_msg	reactions(message_id)	Batch pobieranie reakcji
ft_messages_content	messages(content) FULLTEXT	Wyszukiwanie pelnotekstowe
idx_polls_message	polls(message_id)	Pobieranie ankiet per wiadomosc
idx_user_unread	notifications(user_id, read_at)	Licznik nieprzeczytanych

4. API REST — ENDPOINTY

4.1 Autentykacja

Metoda	Endpoint	Opis
POST	/api/auth/register	Rejestracja nowego uzytkownika
POST	/api/auth/login	Logowanie — zwraca token JWT
GET	/api/auth/me	Dane zalogowanego uzytkownika

4.2 Serwery i kanaly

Metoda	Endpoint	Opis
GET	/api/servers	Lista serwerow uzytkownika
POST	/api/servers	Utworz nowy serwer

GET	/api/servers/:id	Szczegóły serwera
GET	/api/servers/:id/channels	Lista kanałów serwera
POST	/api/servers/:id/channels	Utwórz nowy kanał

4.3 Wiadomości

Metoda	Endpoint	Opis
GET	/api/channels/:id/messages	Pobierz wiadomości (paginacja kursorowa)
POST	/api/channels/:id/messages	Wyślij wiadomość (HTTP fallback)
PATCH	/api/channels/:id/messages/:msgId	Edytuj wiadomość
DELETE	/api/channels/:id/messages/:msgId	Usuń wiadomość
GET	/api/servers/:id/search	Wyszukiwanie FULLTEXT

4.4 Głos

Metoda	Endpoint	Opis
GET	/api/livekit/token	Token dostępu do kanału głosowego
DELETE	/api/livekit/kick	Wyrzucenie użytkownika z głosu

5. KOMUNIKACJA REAL-TIME (SOCKET.IO)

5.1 Zdarzenia klient do serwera

Zdarzenie	Dane	Opis
join_server	serverId	Dolacz do pokoju serwera
join_channel	channelId	Dolacz do pokoju kanału
MESSAGE_CREATE	{channelId, content, nonce}	Wyślij wiadomość
MESSAGE_UPDATE	{messageId, channelId, content}	Edytuj wiadomość
MESSAGE_DELETE	{messageId, channelId}	Usuń wiadomość
REACTION_ADD	{messageId, channelId, emoji}	Dodaj reakcję
TYPING_START	{channelId}	Wskaźnik pisanja

5.2 Zdarzenia serwera do klienta

Zdarzenie	Opis
MESSAGE_CREATE	Nowa wiadomość od innego użytkownika
MESSAGE_UPDATE	Edycja wiadomości

MESSAGE_DELETE	Usuniecie wiadomosci — natychmiastowe u wszystkich
REACTION_UPDATE	Aktualizacja reakcji na wiadomosci
PRESENCE_UPDATE	Zmiana statusu uzytkownika (online/offline)
NOTIFICATION	Nowe powiadomienie (wzmianka, odpowiedz, reakcja)
TYPING_START / TYPING_STOP	Wskaznik pisania w czasie rzeczywistym

6. WYMAGANIA SERWEROWE

6.1 Minimalna specyfikacja per klient (10 uzytkownikow)

Zasob	Minimum	Zalecane
vCPU	2 rdzenie	2 rdzenie
RAM	2 GB	4 GB
Dysk SSD	20 GB	40 GB
Transfer miesieczny	1 TB	Unmetered
Port sieciowy	100 Mbps	1 Gbps

6.2 Zuzycie zasobow per komponent

Komponent	RAM	CPU idle	CPU peak
Node.js + Socket.IO	150-256 MB	2%	15%
MySQL 8.0	256-512 MB	1%	20%
LiveKit SFU	400-512 MB	3%	60%
coturn (TURN/STUN)	64 MB	~0%	5%
System operacyjny	256 MB	2%	—

6.3 Rekomendowane serwery VPS

Dostawca	Model	vCPU	RAM	Cena miesieczna
Hetzner	CAX11 (ARM)	2	4 GB	3.79 EUR
Hetzner	CX22	2	4 GB	4.51 EUR
OVH	Starter-1	1	2 GB	3.50 EUR

7. PROVISIONING — FLOW AUTOMATYCZNY

7.1 Schemat procesu

1. Klient kupuje plan na platformie

2. System billing potwierdza platnosc
3. Provisioning API — POST /v1/servers (Hetzner lub OVH API)
4. Oczekiwanie na status "running" (~20-60 sekund)
5. SSH — wykonanie skryptu instalacyjnego na nowym VPS
6. docker-compose up (Node.js + MySQL + LiveKit + coturn)
7. Konfiguracja nginx + SSL via Let's Encrypt
8. Inicjalizacja bazy danych (schema.sql + migracje)
9. Wyslanie danych dostepowych do klienta (email)
10. Serwer gotowy — czas od zakupu: ponizej 3 minut

7.2 Docker Compose — struktura serwisow

```
version: "3.8"
services:
  backend: # Node.js API + Socket.IO — port 3001
  db: # MySQL 8.0 z wolumenem danych
  livekit: # LiveKit SFU — port 7880, UDP 7881, 50000-60000
  coturn: # TURN/STUN relay — UDP 3478
```

8. BEZPIECZENSTWO

8.1 Autentykacja

- JWT (JSON Web Tokens) z czasem wygasniecia
- bcrypt dla hashowania hasel (salt rounds: 12)
- Tokeny socket weryfikowane przy kazdym polaczeniu

8.2 Autoryzacja

- System rol z granularnym systemem uprawnień (JSON)
- Weryfikacja uprawnień na kazdym endpointzie
- Moderacja: mute, ban, kick z pelnym logowaniem

8.3 Szyfrowanie komunikacji

- HTTPS/WSS (TLS 1.2+) dla wszystkich polaczen HTTP i WebSocket
- DTLS-SRTP dla strumieni audio/wideo (LiveKit)
- Certyfikaty Let's Encrypt z automatycznym odnowieniem

8.4 Izolacja danych

- Kazdy klient = osobny serwer VPS = osobna baza danych
- Brak mozliwosci dostepu cross-tenant
- Dane nie opuszczaja serwera klienta (poza frontendem Vercel)

8.5 Ochrona przed atakami

- Rate limiting na endpointach API
- Walidacja typow i rozmiarow uploadowanych plikow (max 8 MB)

- Sanityzacja danych wejsciowych
- DDoS protection przez dostawce VPS (Hetzner/OVH)

9. SKALOWANIE

9.1 Skalowanie poziome (wiele instancji Node.js)

- Dodanie Redis jako adaptera Socket.IO dla sesji wspoldzielonych
- Load balancer nginx z upstream do wielu instancji
- MySQL connection pooling dla optymalnego wykorzystania polaczen

9.2 Storage plikow

Aktualne rozwiazanie: pliki przechowywane w MySQL jako MEDIUMBLOB — wystarczajace do ~100 GB per instancja.

Planowane: migracja do Cloudflare R2 (object storage z darmowym egress) — redukcja obciazenia bazy i kosztow dysku.

9.3 Progi skalowania

Liczba uzytkownikow	Zalecana konfiguracja
10	CX22 / CAX11 — konfiguracja standardowa
50	CX32 (4 vCPU, 8 GB RAM)
200	CX52 (8 vCPU, 16 GB RAM)
500+	Dedykowany serwer + Redis + MySQL repliki

10. MONITORING I DOSTEPNOSC

10.1 Metryki do monitorowania

- Uzycie CPU i RAM — prog alertu przy >80%
- Liczba aktywnych polaczen WebSocket
- Rozmiar bazy danych i wolnego miejsca na dysku
- Czas odpowiedzi API (p95 ponizej 200 ms)
- Dostepnosc portu UDP dla glosu (LiveKit)
- Aktywne sesje glosowe per klient

10.2 SLA i dostepnosc

Parametr	Wartosc
Cel dostepnosci	99.9% (< 9 godzin przestoju rocznie)
Backup bazy danych	Co 24 godziny, retencja 7 dni
Czas przywrocenia (RTO)	Ponizej 2 godzin

Punkt przywrócenia (RPO)	Maksymalnie 24 godziny
--------------------------	------------------------